Module IV

Semantic Analysis and Intermediate code generation

Course Outcome

CO4

- Build Syntax Directed Translation for a context free grammar, compare various storage allocation strategies and classify intermediate representations
- **Cognitive Knowledge Level: Apply**

Need for Semantic Analysis

- Semantic analysis is a phase by a compiler that adds semantic information to the parse tree and performs certain checks based on this information.
- It logically follows the parsing phase, in which the parse tree is generated, and logically precedes the code generation phase, in which code is generated.
- Typical examples of semantic information that is added and checked is typing information - type checking, and the binding of variables and function names to their definitions - object binding.
- Sometimes also some early code optimization is done in this phase. For this phase the compiler usually maintains symbol tables in which it stores what each symbol refers to.

Semantic Analysis - Tasks

1. Disambiguate Overloaded operators : If an operator is overloaded, one would like to specify the meaning of that particular operator because from one will go into code generation phase next.
2. Type checking : The process of verifying and enforcing the constraints of types is called type checking.
   - This may occur either at compile-time (static check) or run-time (dynamic check).
   - Static type checking is a primary task of the semantic analysis carried out by a compiler.
   - If type rules are enforced strongly (that is, generally allowing only those automatic type conversions which do not lose information), the process is called strongly typed, if not, weakly typed.

Semantic Analysis - Tasks

3. Uniqueness checking : Whether a variable name is unique or not, in the its scope.

4. Type coercion : If some kind of mixing of types is allowed. Done in languages which are not strongly typed. This can be done dynamically as well as statically.

Following things are done in Semantic Analysis

5. Name Checks : Check whether any variable has a name which is not allowed. Ex. Name is same as an identifier(Ex. int in java).
   - A parser has its own limitations in catching program errors related to semantics, something that is deeper than syntax analysis.
   - Typical features of semantic analysis cannot be modeled using context free grammar formalism.
   - If one tries to incorporate those features in the definition of a language then that language doesn't remain context free anymore.
   - These are a couple of examples which tell us that typically what a compiler has to do beyond syntax analysis.
   - An identifier x can be declared in two separate functions in the program, one of the type int and then of the type char. Hence the same identifier will have to be bound to these two different properties in the two different contexts.

Semantic Errors

- We have mentioned some of the semantics errors that the semantic analyzer is expected to recognize:
  - Type mismatch
  - Undeclared variable
  - Reserved identifier misuse.
  - Multiple declaration of variable in a scope.
  - Accessing an out of scope variable.
  - Actual and formal parameter mismatch.

Syntax Directed Translation

- The Principle of Syntax Directed Translation states that the meaning of an input sentence is related to its syntactic structure, i.e., to its Parse-Tree. By Syntax Directed Translations we indicate those formalisms for specifying translations for programming language constructs guided by context- free grammars.
  - We associate Attributes to the grammar symbols representing the language constructs.
  - Values for attributes are computed by Semantic Rules associated with grammar productions.

Syntax Directed Translation

- Evaluation of Semantic Rules may:
  - Generate Code;
  - Insert information into the Symbol Table;
  - Perform Semantic Check;
  - Issue error messages;
  - etc.
- There are two ways to represent the semantic rules associated with grammar symbols.
  1. Syntax-Directed Definitions (SDD)
  2. Syntax-Directed Translation Schemes (SDT)

Syntax Directed Definitions

- Syntax Directed Definition|Compiler Design|Part# 31|Learn CSE Malayalam

Syntax Directed Definitions

- Syntax Directed Definitions are a generalization of context-free grammars in which:
  1. Grammar symbols have an associated set of Attributes;
  2. Productions are associated with Semantic Rules for computing the values of attributes.
- Such formalism generates Annotated Parse-Trees where each node of the tree is a record with a field for each attribute (e.g., X.a indicates the attribute a of the grammar symbol X).
  - The value of an attribute of a grammar symbol at a given parse-tree node is defined by a semantic rule associated with the production used at that node.

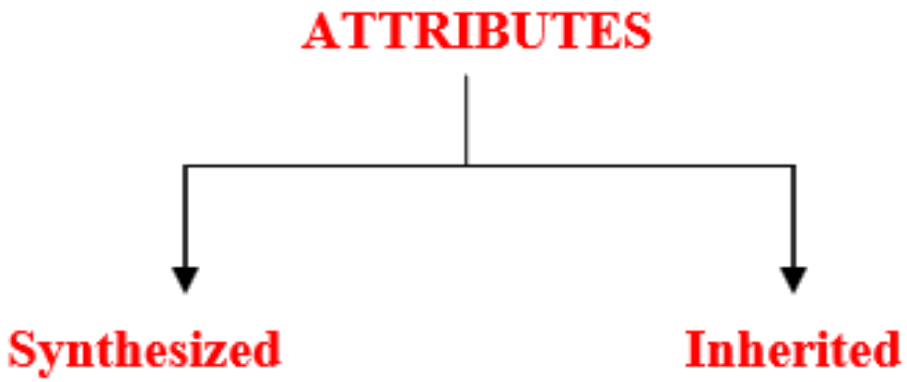
Attribute Grammar

- Attributes are properties associated with grammar symbols. Attributes can be numbers, strings, memory locations, datatypes, etc
- Attribute grammar is a special form of context-free grammar where some additional information (attributes) are appended to one or more of its non-terminals in order to provide context-sensitive information.
- Attribute grammar is a medium to provide semantics to the context-free grammar and it can help specify the syntax and semantics of a programming language. Attribute grammar (when viewed as a parse-tree) can pass values or information among the nodes of a tree.

Eg: $E \rightarrow E + T \{ E.value = E.value + T.value \}$

Attribute Grammar

- The right part of the CFG contains the semantic rules that specify how the grammar should be interpreted. Here, the values of non-terminals E and T are added together and the result is copied to the non-terminal E.
- Semantic attributes may be assigned to their values from their domain at the time of parsing and evaluated at the time of assignment or conditions.
- Based on the way the attributes get their values, they can be broadly divided into two categories : synthesized attributes and inherited attributes



Synthesized Attributes

- These are those attributes which get their values from their children nodes i.e. value of synthesized attribute at node is computed from the values of attributes at children nodes in parse tree.

  To illustrate, assume the following production:

  EXAMPLE : S -> ABC

  S.a= A.a,B.a,C.a

- If S is taking values from its child nodes (A,B,C), then it is said to be a synthesized attribute, as the values of ABC are synthesized to S.

Computation of Synthesized Attributes

- Write the SDD using appropriate semantic rules for each production in given grammar.
- The annotated parse tree is generated and attribute values are computed in bottom up manner (LR Parser)
- The value obtained at root node is the final output.

Computation of Synthesized Attributes

$S \rightarrow E$
$E \rightarrow E1 + T$
$E \rightarrow T$
$T \rightarrow T1 * F$
$T \rightarrow F$
$F \rightarrow digit$

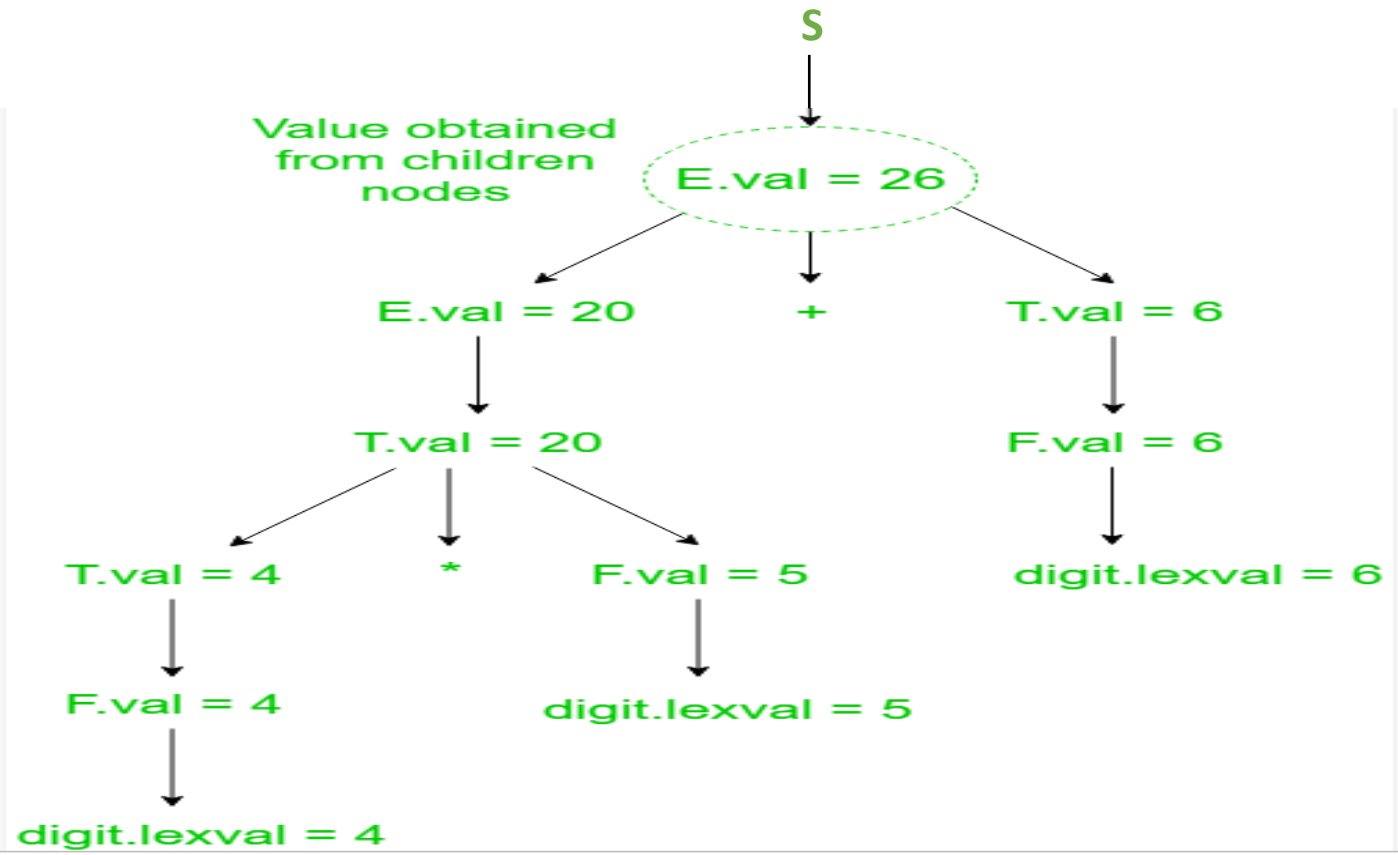
Computation of Synthesized Attributes

- The SDD for the above grammar can be written as follow

PRODUCTIONS	SEMANTIC RULES
S --> E	Print(E.val)
E --> E_1 + T	E.val = E_1.val + T.val
E --> T	E.val = T.val
T --> T_1 * F	T.val = T_1.val * F.val
T --> F	T.val = F.val
F --> digit	F.val = digit.lexval

Computation of Synthesized Attributes

- Let us assume an input string **4 * 5 + 6** for computing synthesized attributes. The annotated parse tree for the input string is



Computation of Synthesized Attributes

- For computation of attributes we start from leftmost bottom node. The rule F –> digit is used to reduce digit to F and the value of digit is obtained from lexical analyzer which becomes value of F i.e. from semantic action F.val = digit.lexval.
- Hence, F.val = 4 and since T is parent node of F so, we get T.val = 4 from semantic action T.val = F.val.
- Then, for T –> T1 * F production, the corresponding semantic action is T.val = T1.val * F.val . Hence, T.val = 4 * 5 = 20
- Similarly, combination of E1.val + T.val becomes E.val i.e. E.val = E1.val + T.val = 26. Then, the production S –> E is applied to reduce E.val = 26 and semantic action associated with it prints the result E.val . Hence, the output will be 26.

Annotated parse tree

- The parse tree containing the values of attributes at each node for given input string is called annotated or decorated parse tree.

Inherited Attributes

- These are the attributes which inherit their values from their parent or sibling nodes. i.e. value of inherited attributes are computed by value of parent or sibling nodes.

EXAMPLE:

A --> BCD { C.in = A.in, C.type = B.type }

- B can get values from A, C and D. C can take values from A, B, and D. Likewise, D can take values from A, B, and C.

Computation of Inherited Attributes

- Construct the SDD using semantic actions.
- The annotated parse tree is generated and attribute values are computed in top down manner (LL Parser).
- Consider the following grammar

Computation of Inherited Attributes

S → T L

T → int

T → float

T → double

L → L1, id

L → id

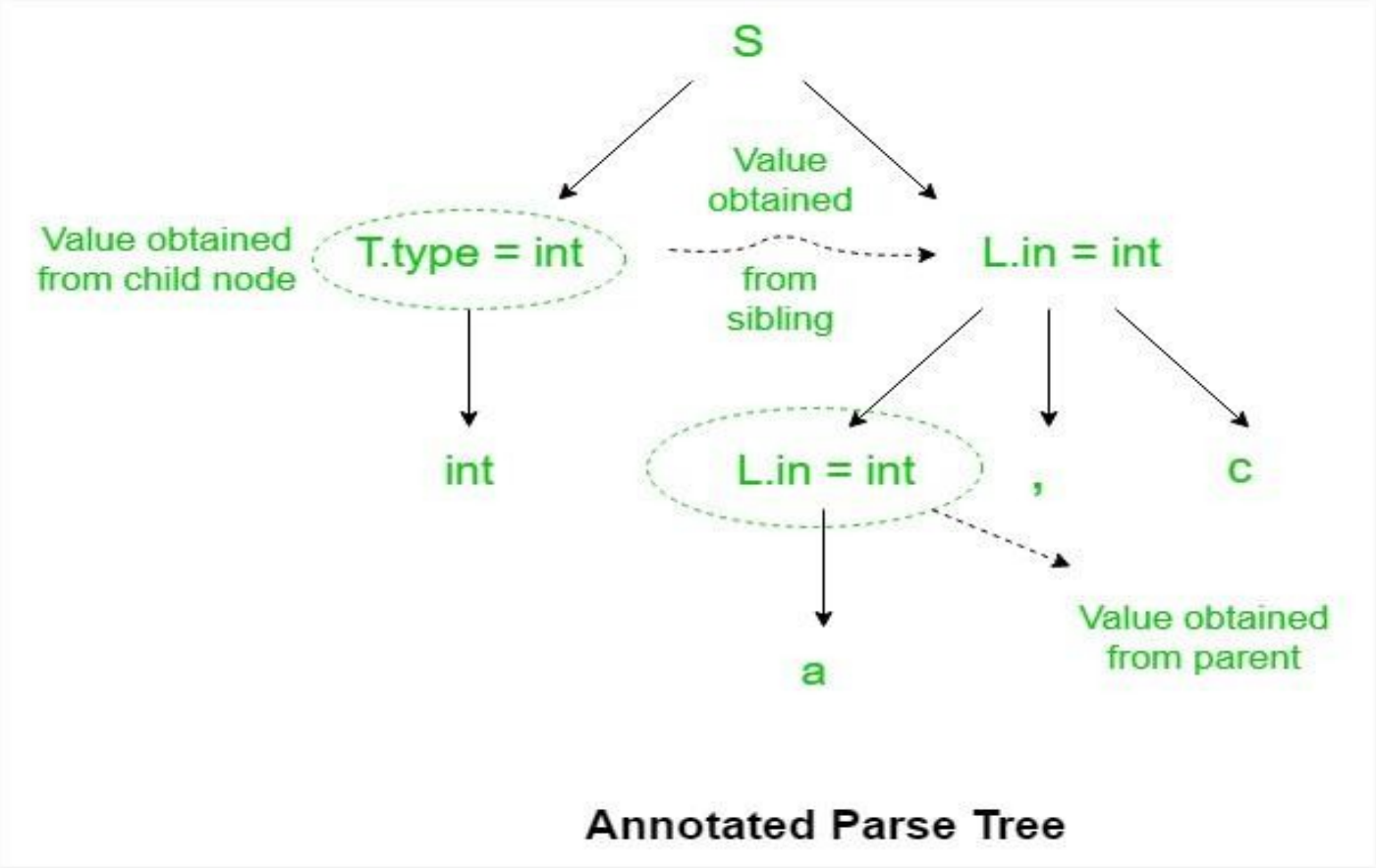
Computation of Inherited Attributes

- The SDD for the above grammar can be written as follow

Production	Semantic Actions
S --> T L	L.in = T.type
T --> int	T.type = int
T --> float	T.type = float
T --> double	T.type = double
L --> L_1 , id	L_1.in = L.in Enter_type(id.entry , L.in)
L --> id	Entry_type(id.entry , L.in)

Computation of Inherited Attributes

- Let us assume an input string **int a, c** for computing inherited attributes. The annotated parse tree for the input string is



Computation of Inherited Attributes

- The value of L nodes is obtained from T.type (sibling) which is basically lexical value obtained as int, float or double.
- Then L node gives type of identifiers a and c. The computation of type is done in top down manner or preorder traversal.
- Using function Enter_type the type of identifiers a and c is inserted in symbol table at corresponding id.entry

Implementing Syntax Directed Definitions

1. Dependency Graphs
   - Implementing a Syntax Directed Definition consists primarily in finding an order for the evaluation of attributes
   - Each attribute value must be available when a computation is performed. Dependency Graphs are the most general technique used to evaluate syntax directed definitions with both synthesized and inherited attributes.
   - Annotated parse tree shows the values of attributes, dependency graph helps to determine how those values are computed

Dependency Graphs

- Dependency Graph|Compiler Design|Part# 32|Learn CSE Malayalam

Dependency Graphs

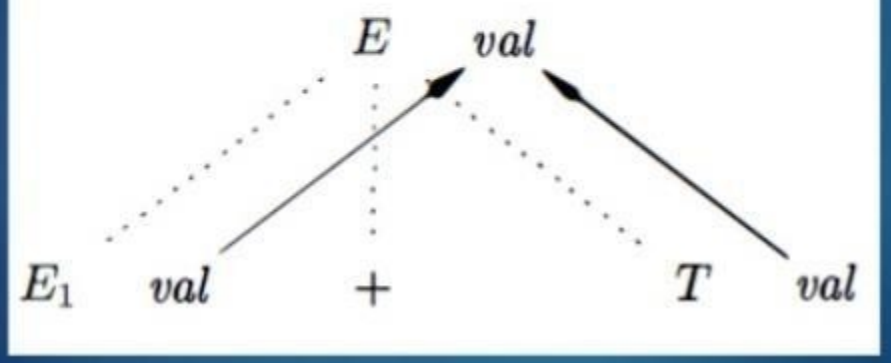
- The interdependencies among the attributes of the various nodes of a parse-tree can be depicted by a directed graph called a dependency graph.
- There is a node for each attribute;
- If attribute ‘b’ depends on an attribute ‘c’ there is a link from the node for c to the node for b ($b \leftarrow c$).

Dependency Rule:

- If an attribute b depends from an attribute c, then we need to find the semantic rule for c first and then the semantic rule for b.

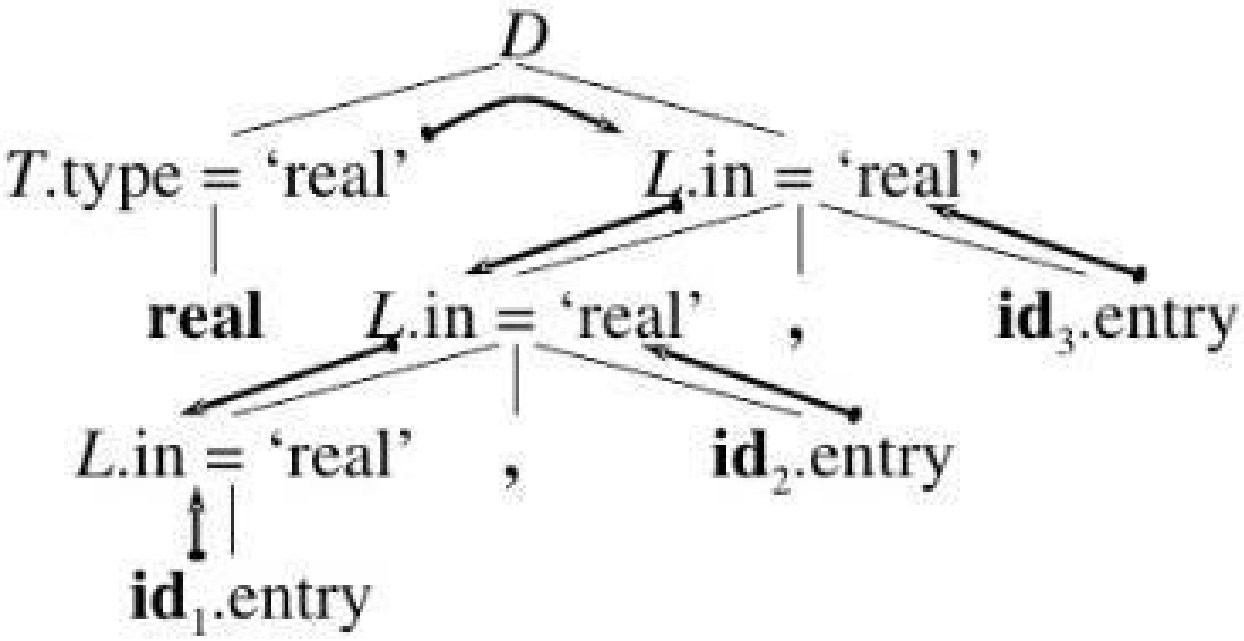
Dependency Graphs

PRODUCTION	SEMANTIC RULE
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$



Dependency Graphs

- Dependency graph for declaration statements.



Implementing Syntax Directed Definitions

2. Evaluation order

- A dependency graph characterizes the possible order in which we can calculate the attributes at various nodes of the parse tree.
- If there is an edge from node M to N, then the attribute corresponding to M first be evaluated before evaluating N.
- Thus the allowable orders of evaluation are N1,N2…..,Nk such that if there is an edge from Ni to Nj then i<j
- Such an ordering embeds a directed graph into a linear order, and is called a topological sort of the graph.
- If there is any cycle in the graph ,then there is no topological sort.ie, there is no way to evaluate SDD on this parse tree.

Syntax Directed Translation

- Syntax Directed Translation|Compiler Design|Part# 33|Learn CSE Malayalam

TYPES OF SDT'S

1. S –attributed definition
2. L –attributed definition

S-attributed definition

- S stands for synthesized
- If an SDT uses only synthesized attributes, it is called as S-attributed SDT.

EXAMPLE:

$A \rightarrow BC \quad \{ A.a=B.a,C.a\}$

- S-attributed SDTs are evaluated in bottom-up parsing, as the values of the parent nodes depend upon the values of the child nodes.
- Semantic actions are placed in rightmost place of RHS.

A -> BCD { }

L –attributed definition

- L stands for one parse from left to right.

- **L-Attributed Definitions** contain both synthesized and inherited attributes but do not need to build a dependency graph to evaluate them.

- **Definition.** A syntax directed definition is *L-Attributed* if each *inherited attribute* of X_j in a production $A \rightarrow X_1 \ldots X_j \ldots X_n$, depends only on:

  1. The attributes of the symbols to the **left** (this is what L in *L-Attributed* stands for) of X_j, i.e., $X_1 X_2 \ldots X_{j-1}$, and
  2. The *inherited* attributes of A.

L –attributed definition

- ie, If an SDT uses both synthesized attributes and inherited attributes with a restriction that inherited attribute can inherit values from parent and left siblings only, it is called as L-attributed SDT.

EXAMPLE:

- A → BCD {B.a=A.a, C.a=B.a}
  {C.a=D.a} This is not possible
- Attributes in L-attributed SDTs are evaluated by depth-first and left-to-right parsing manner.
- **If an attribute is S attributed , it is also L attributed.**

L –attributed definition

- **L-Attributed Definitions are a class of syntax directed definitions whose attributes can always be evaluated by single traversal of the parse-tree.**
- The following procedure evaluate L-Attributed Definitions by mixing PostOrder (synthesized) and PreOrder (inherited) traversal.

Algorithm: **L-Eval(n: Node)**

Input: Node of an annotated parse-tree.

Output: Attribute evaluation.

```
Begin
        For each child m of n, from left-to-right Do
        Begin
                Evaluate inherited attributes of m;
                L-Eval(m)
        End;
        Evaluate synthesized attributes of n
End.
```

SDD for desk calculator/SDD for evaluation of expressions

	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E$ **n**	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow$ **digit**	$F.val =$ **digit**.lexval

SSD for a desk calculator

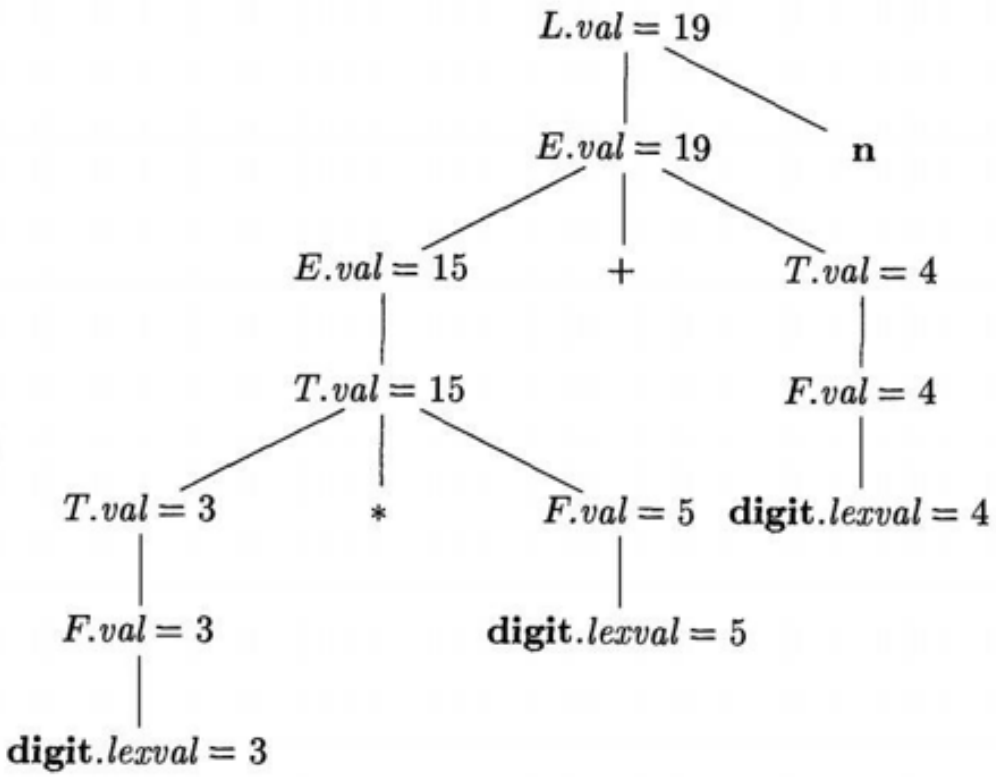
SDD for desk calculator/SDD for evaluation of expressions

- Evaluate the expression 3*5+4n using the above SDD both in bottom up and top down approach

Solution:

- In both case first we need to draw the parse tree.
- Then traverse from top to down left to right
- In bottom up approach, whenever there is a reduction ,go to the production and carry out the action

SDD for desk calculator/SDD for evaluation of expressions



Annotated parse tree for $3 * 5 + 4\,\mathbf{n}$

SDD for desk calculator/SDD for evaluation of expressions

Solution: Top down approach:

- First we need to draw the parse tree.
- Then traverse from top to down left to right
- In Top down up approach, whenever we come across a semantic action, it is performed

SDD for desk calculator/SDD for evaluation of expressions

Production	Semantic Rules
$T \rightarrow FT'$	$T'.inh = F.val$
	$T.val = T'.syn$
$T' \rightarrow *FT_1'$	$T_1'.inh = T'.inh \times F.val$
	$T'.syn = T_1'.syn$
$T' \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

SDD for desk calculator/SDD for evaluation of expressions

- Evaluate the expression 3*5 using the above SDD both in bottom up and top down approach

Syntax Directed Translation Scheme

- Syntax directed translation scheme(SDT) is a context free grammar with program fragments embedded within production bodies.
- The program fragments are called semantic actions and can appear at any position within a production body.

$A \rightarrow \{\ \}BC$

$A \rightarrow B\{\ \}C$

$A \rightarrow BC\{\ \}$

Syntax Directed Translation Scheme

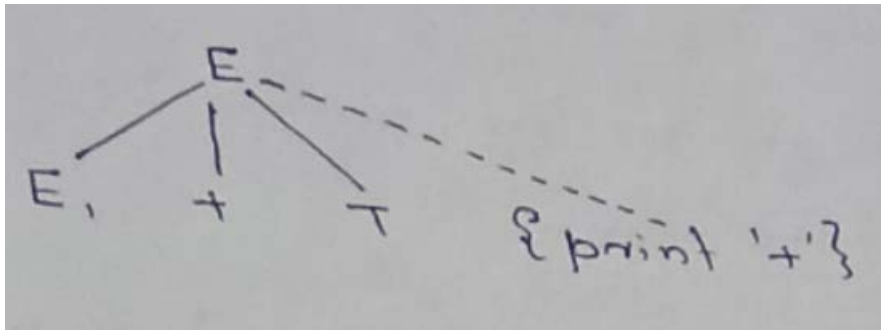
- By convention, we place curly braces around actions, if braces are need as grammar symbols, then we quote them.

Eg, $E \rightarrow E_1 + T$ {print '+'}

Eg: {print '{' }

Syntax Directed Translation Scheme

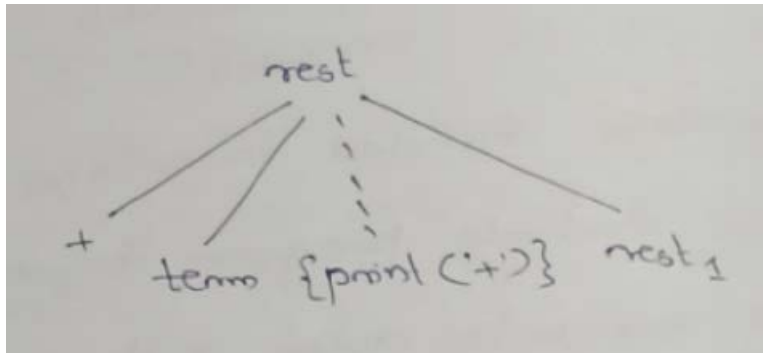
- Any SDT can be implemented by first building a parse tree and then performing the actions in a left to right depth first order, that is during a pre order traversal.
- An extra leaf is constructed for a semantic action.



Syntax Directed Translation Scheme

- In translation scheme, the sematic action may occur at any position in a production body.

Eg: rest → + term { print('+') } rest1



Syntax Directed Translation Scheme

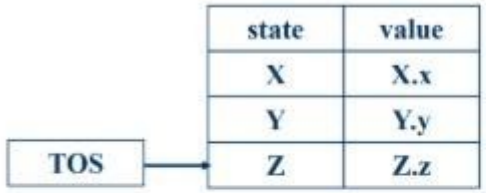
- When drawing a parse tree in translation scheme, we indicate an action by constructing an extra child for it, connected by a dashed line to the node that corresponds to the head of the production.
- In B → X {a} Y, the action 'a' is done after we have recognized X, if X is a terminal or all the terminals derived from X if X is a non terminal.
- If the parse is bottom up then we perform action 'a' as soon as this occurrence of X appear on the top of the parsing stack.
- If the parse is top down we perform 'a' just before we attempt to expand this occurrence of Y(non terminal) or check for Y on the input (terminal).

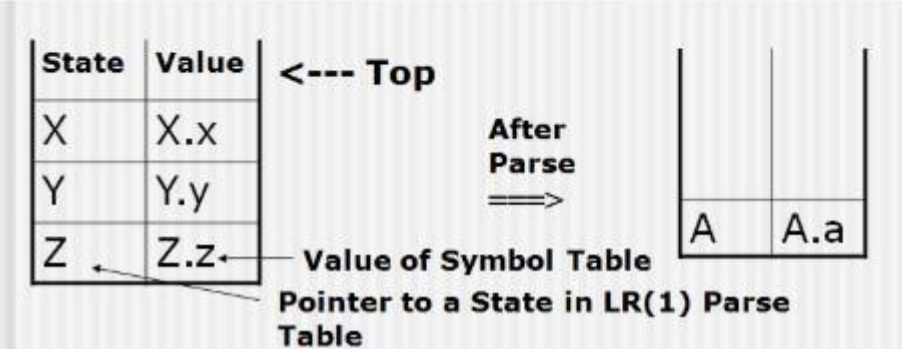
Syntax Directed Translation Scheme

- SDT's with all actions at the right end of the production bodies are called postfix SDT's
- Postfix SDT's can be implemented during LR parsing by executing the actions when reduction occurs.
- The attribute of each grammar symbol can be put on the stack in a place where they can be found during the reduction.
- The best plan is to place the attribute along with the grammar symbols in records on the stack itself.

Syntax Directed Translation Scheme

- e.g. A => XYZ and A.a := f (X.x, Y.y, Z.z)





Syntax Directed Translation Scheme

- Syntax directed translation schemes are a complementary notation to syntax directed definitions.
- Between the two notations, syntax directed definition can be more readable and hence more useful for specifications.
- The translation scheme can be more efficient and hence more useful for implementation.

Syntax Directed Translation Scheme

The rules for turning an L attributed SDD into an SDT are:

1. Embed the action that computes the inherited attributes for a nonterminal A immediately before that occurrence of A in the body of the production.
   - If several inherited attributes for A depend on one another in an acyclic fashion, order the evaluation of attributes so that those needed first are computed first.
2. Place the actions that compute a synthesized attribute for the head of a production at the end of the body of that production.

Syntax Directed Translation Scheme

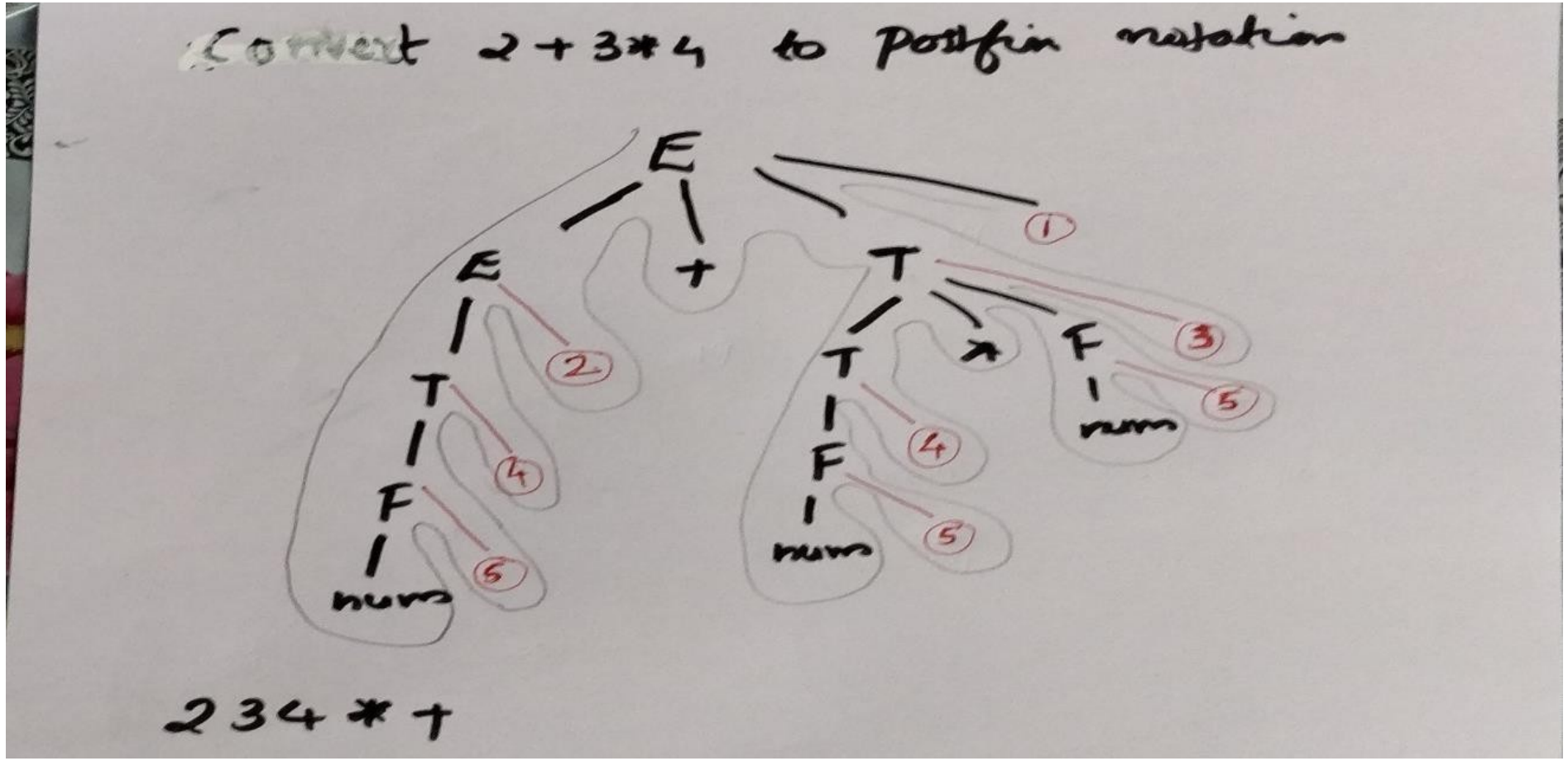
- Construction of Syntax Trees|Compiler Design|Part# 34|Learn CSE Malayalam

Infix to Postfix conversion Grammar

- GRAMMAR SEMANTIC ACTIONS

$E \rightarrow E + T$	{ Print ("+") }	1
$E \rightarrow T$	{ }	2
$T \rightarrow T * F$	{ Print ("*") }	3
$T \rightarrow F$	{ }	4
$F \rightarrow num$	{ Print (num.lvalue) }	5

Infix to Postfix conversion Grammar

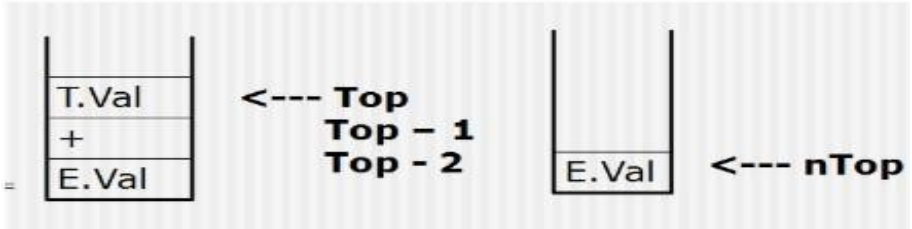


Calculator

- Consider again the syntax-directed definition of the desk calculator

PRODUCTION	CODE FRAGMENT
$L \rightarrow E\ \mathbf{n}$	$print(val[top])$
$E \rightarrow E_1 + T$	$val[ntop] := val[top-2] + val[top]$
$E \rightarrow T$	
$T \rightarrow T_1 * F$	$val[ntop] := val[top-2] \times val[top]$
$T \rightarrow F$	
$F \rightarrow (\ E\ )$	$val[ntop] := val[top-1]$
$F \rightarrow \mathbf{digit}$	

Fig. 5.16. Implementation of a desk calculator with an LR parser.



- Example :- SDD and code fragment using S attributed definition for the input 3*5+4n is as follows:

Production	Semantic Rule	Code fragment
$L \rightarrow En$	$L.val = E.val$ or $Print(E.val)$	$Print(val[top])$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	$val[ntop] = val[top-2] + val[top]$
$E \rightarrow T$	$E.val = T.val$	
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$	$val[ntop] = val[top-2] * val[top]$
$T \rightarrow F$	$T.val = F.val$	
$F \rightarrow (E)$	$F.val = E.val$	$val[ntop] = val[top-1]$
$F \rightarrow digit$	$F.val = digit.lvalue$.	

Semantic Analyser

Bottom up evaluation of S attributed definitions

- Bottom up Evaluation of S attributed Definitions|Compiler Design|Part# 35|Learn CSE Malayalam

Bottom up evaluation of S attributed definitions

Eg: Moves made by translator on input 3 * 5 + 4n

Input	State	Value	Production
3*5+4n	-	-	-
*5+4n	3	3	-
*5+4n	F	3	F → digit
*5+4n	T	3	T → F
5+4n	T *	3_	-
+4n	T * 5	3_5	-
+4n	T * F	3_5	F → digit
+4n	T	15	T → T * F

Bottom up evaluation of S attributed definitions

Eg: Moves made by translator on input 3 * 5 + 4n

Input	State	Value	Production
+4n	E	15	E → T
4n	E +	15 _	-
n	E + 4	15 _ 4	-
n	E + F	15_4	F → digit
n	E + T	15 _ 4	T → digit
n	E	19	E → E + T
-	En	19	-
-	L	19	L → En

Bottom up evaluation of inherited attributes

- Bottom up Evaluation of Inherited attributes|Compiler Design|Part# 36|Learn CSE Malayalam

Bottom up evaluation of inherited attributes

- L attributed definitions are the syntax directed definition with both inherited and synthesized attributes. (LL(1) Parser)
- A bottom up parser reduces the right side of production A → XY by removing X and Y from the top of the parser stack and replacing them by A.
- Suppose X has a synthesized attribute X.s and Y has an inherited attribute Y.i, then the value of X.s is already on the parser stack before any reductions take place in the subtree below Y, and this value can be inherited by Y.

Bottom up evaluation of inherited attributes

- If inherited attribute Y.i is defined by the copy rule Y.i = X.s then the value X.s can be used where Y.i is called for.
- Copy rule play an important role in the evaluation of inherited attribute during bottom up parsing.

Bottom up evaluation of inherited attributes

Eg: Syntax directed definition for simple type declarations

Production	Semantic Rules
D → TL	L.inh = T.type
T → int	T.type = integer
T → float	T.type = float
L → L_1, id	L_1.inh = L.inh
	addType(id.entry, L.inh)
L → id	addType(id.entry, L.inh)

Where, D – declaration
T – type
L – list of identifiers

addType() insert values to symbol table

Bottom up evaluation of inherited attributes

Eg: float p, q, r

Input	State	Production
float p, q, r	-	-
p, q, r	float	-
p, q, r	T	T → float
, q, r	T p	-
, q, r	T L	L → id
q, r	T L ,	-
, r	T L, q	-
, r	T L	L → L, id
r	T L,	-
-	T L, r	-
-	T L	L → L, id
-	D	D → T L

Runtime Environment

Runtime Environment

- Run Time Environments: Source Language Issues|Compiler Design|Part# 39|Learn CSE Malayalam

Runtime Environment

- A translation needs to relate the static source text of a program to the dynamic actions that must occur at runtime to implement the program.
- The program consists of names for procedures, identifiers etc., that require mapping with the actual memory location at runtime.
- Runtime environment is a state of the target machine, which may include software libraries, environment variables, etc., to provide services to the processes running in the system.

Runtime Environment

- The allocation and deallocation of data objects is managed by run tim support package

Source Language Issues

- A ***procedure*** definition is a declaration that associates an identifier with a statement.
- The identifier is ***procedure*** name, and statement is the ***procedure*** body.

```
procedure readarray;
var i : integer;
begin
    for i : = 1 to 9 do read(a[i])
end;
```

Activation Tree

- Each execution of procedure is referred to as an activation of the procedure. Lifetime of an activation is the sequence of steps present in the execution of the procedure.
- If ‘a’ and ‘b’ be two procedures, then their activations will be non-overlapping (when one is called after other) or nested (nested procedures).
- A procedure is recursive if a new activation begins before an earlier activation of thesame procedure has ended. An activation tree shows the way control enters and leaves, activations.

Activation Tree

Properties of activation trees are :

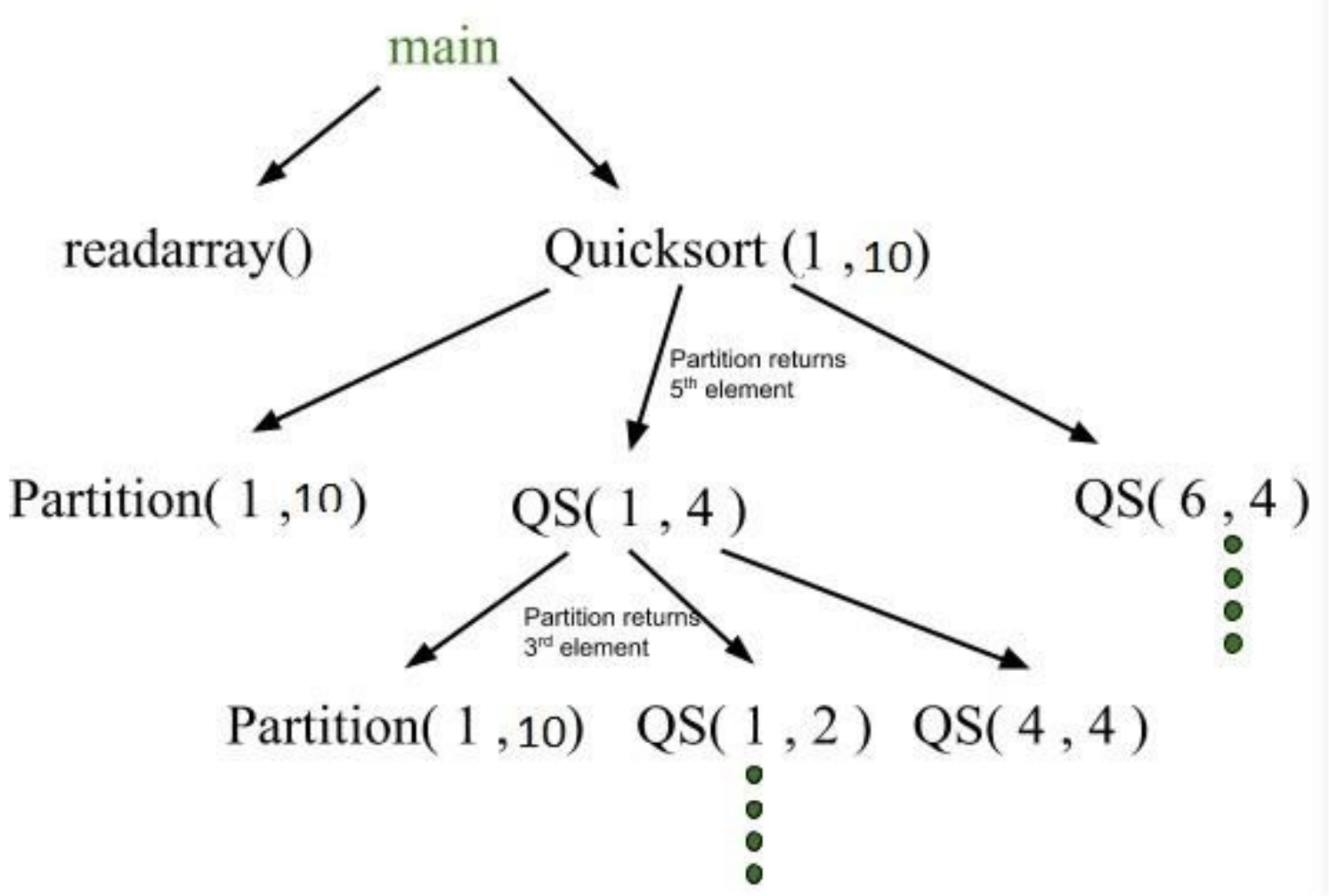
- Each node represents an activation of a procedure.
- The root shows the activation of the main function.
- The node for procedure ‘x’ is the parent of node for procedure ‘y’ if and only if the control flows from procedure x to procedure y.

Example

- Consider the following program of quicksortmain()

```
main();
{
readarray();
quicksort(1,10);
}
quicksort(int m, int n)
{
int i= partition(m,n);
quicksort(m,i-1);
quicksort(i+1,n);
}
```

Activation Tree



Control Stack

- Control stack or runtime stack is used to keep track of the live procedure activations
- i.e the procedures whose execution have not been completed.
- A procedure name is pushed on to the stack when it is called (activation begins) and it is popped when it returns (activation ends).
- Information needed by a single execution of a procedure is managed using an activation record

Control Stack

quicksort(4,4)
Quicksort(1,4)
Quicksort(1.10)
Main()

Scope of a declaration

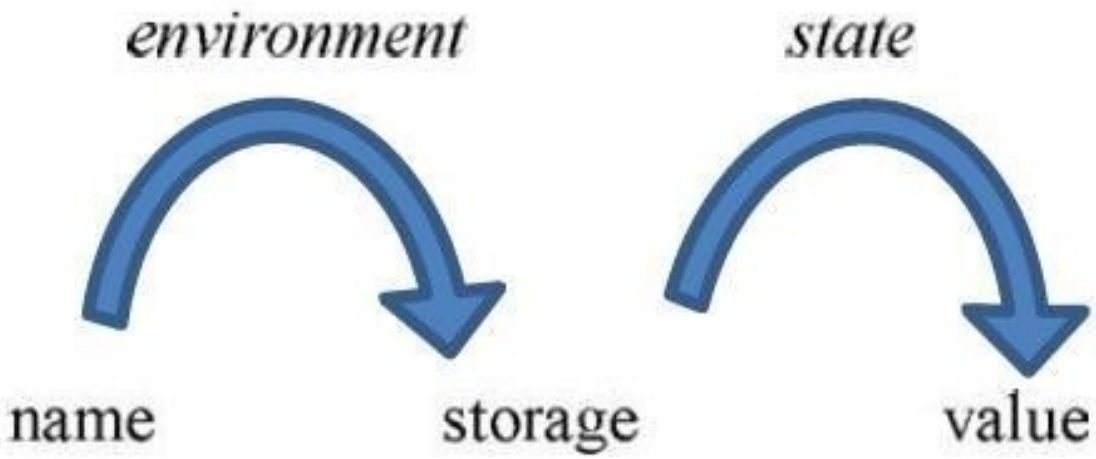
- A declaration is a syntactic construct that associates information with a name.
- Declaration may be explicit such as

var i : integer;

Binding of Names

- Even if each name is declared once in a program, the same name may denote different data object at run time. “Data objects” corresponds to a storage location that hold values.
- The term environment refers to a function that maps a name to a storage location.
- The term state refers to a function that maps a storage location to the value held there.

Binding of Names



Storage Organization

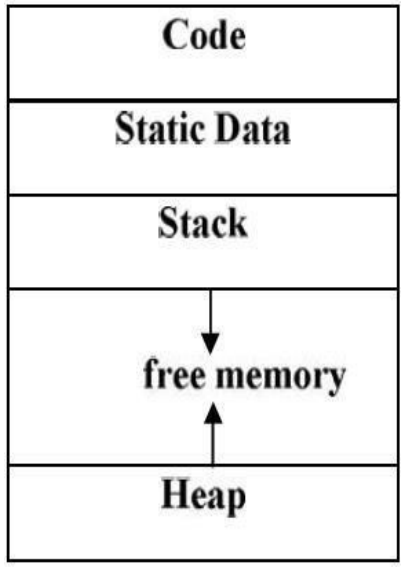
- Run Time Environments: Storage Organization|Compiler Design|Part# 40|Learn CSE Malayalam

Storage Organization

- Typical subdivision of run time memory.
  - Target Code
  - Data objects
  - Control Stack

Storage Organization

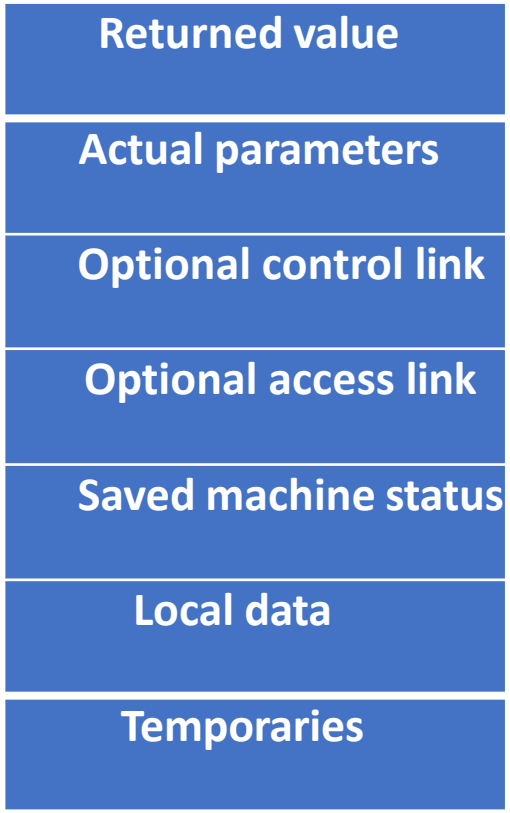
- Typical subdivision of run time memory.



Activation Records

- It is LIFO structure used to hold information about each instantiation.
- Procedure calls and returns are usually managed by a run time stack called control stack.
- Each live activation has an activation record on control stack, with the root of theactivation tree at the bottom, the latter activation has its record at the top of the stack

Activation Record



Activation Records

- Temporary values, such as those arising in the evaluation of expressions, are stored in the field for temporaries.
- The field for local data holds data that is local to an execution of a procedure.
- The field for saved machine status holds information about the state of the machine just before the procedure is called. This information includes the values of the program counter and machine registers that have to be restoredwhen control returns from the procedure.

Activation Records

- The optional access link is used to refer to nonlocal data held in other activation records.
- The optional control link points to the activation record of the caller
- The field for actual parameters is used by the calling procedure to supply parameters to the called procedure.
- The field for the returned value is used by the called procedure to return a value to the calling procedure, Again, in practice this value is often returned in a register for greater efficienc

Storage Allocation Strategies

- Run Time Environments: Storage Allocation Strategies|Compiler Design|Part# 41|Learn CSE Malayalam

Storage Allocation Strategies

- The different storage allocation strategies are:
- Static allocation - lays out storage for all data objects at compile time
- Stack allocation - manages the run-time storage as a stack.
- Heap allocation - allocates and deallocates storage as needed at run time from a data area known as heap.

Static Allocation

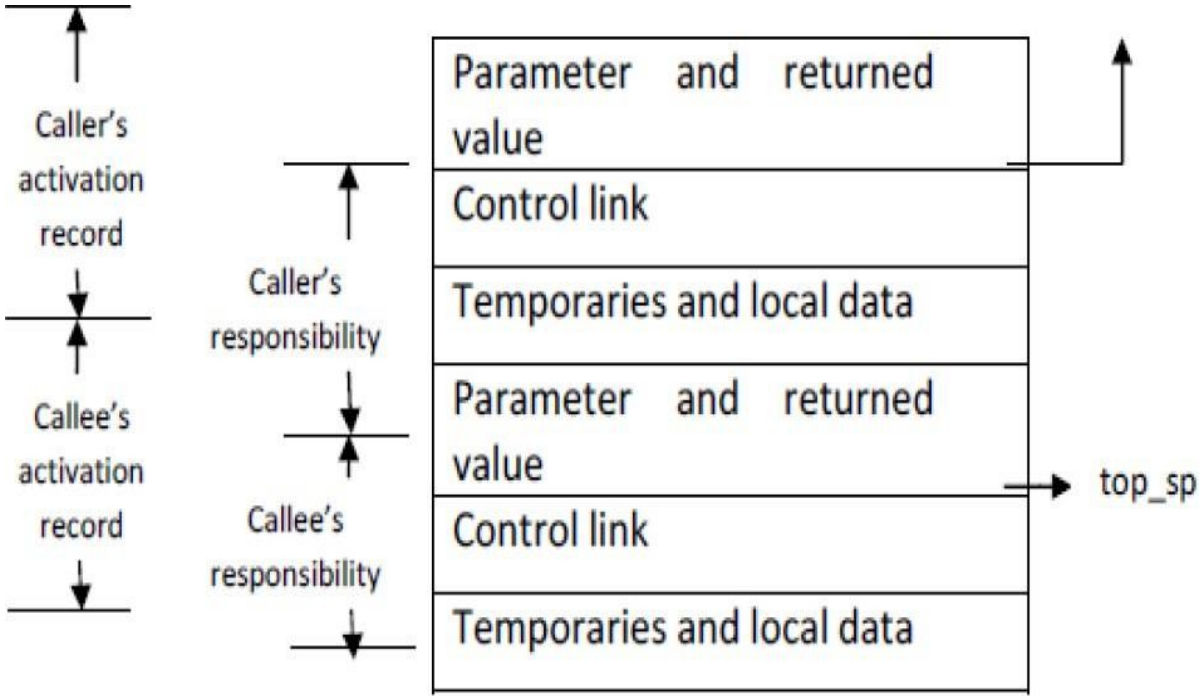
- In static allocation, names bound to storage as the program is compiled, so there is no need for a run-time support package.
- Since the bindings do not change at runtime, every time a procedure activated, its run- time, names bounded to the same storage location.
- Therefore, values of local names retained across activations of a procedure. That is when control returns to a procedure the value of the local are the same as they were when control left the last time.
- From the type of a name, the compiler decides amount of storage for the name and decides where the activation records go. At compile time, we can fill in the address at which the target code can find the data it operates on.

Stack Allocation

- All compilers for languages that use procedures, functions or methods as units of user functions define actions manage at least part of their runtime memory as a stack run- time stack
- Each time a procedure called, space for its local variables is pushed onto a stack, andwhen the procedure terminates, space popped off from the stack

Stack Allocation

- Calling Sequence



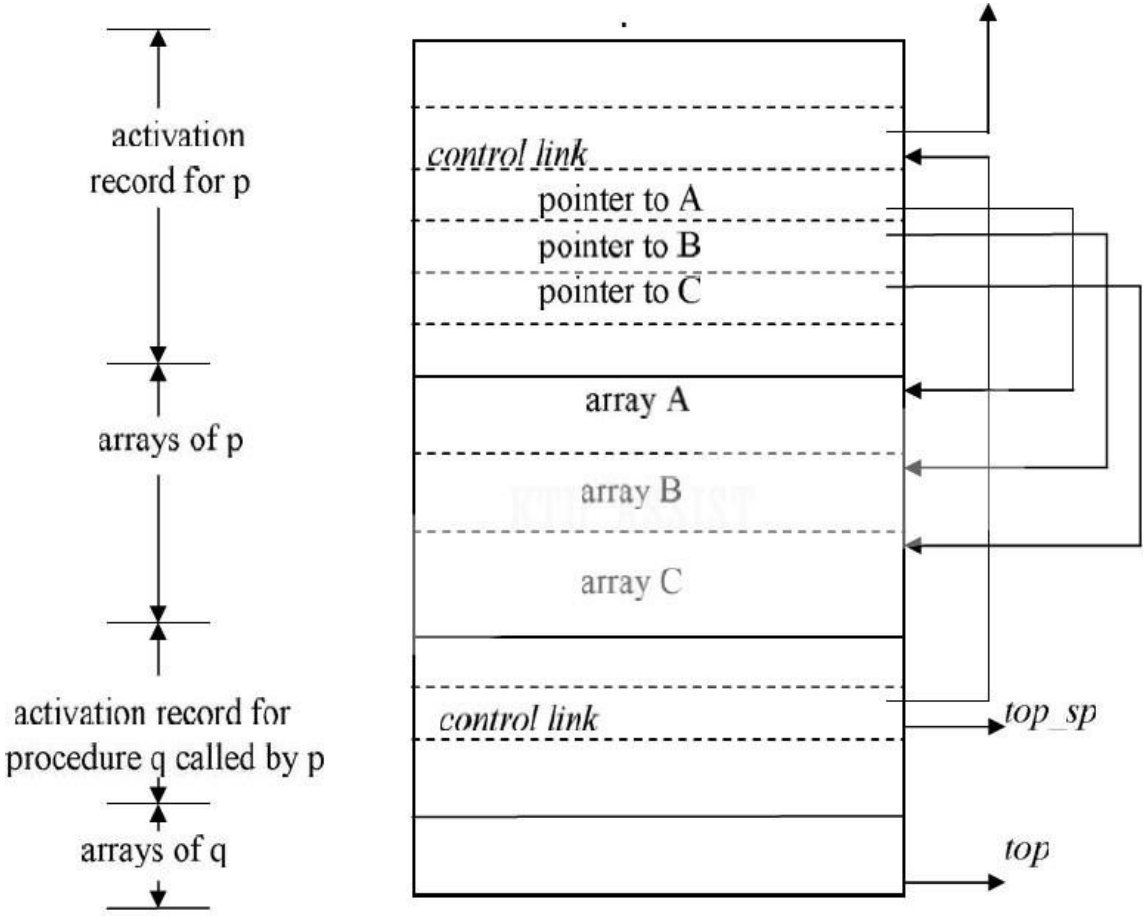
Stack Allocation

Dangling reference

- Whenever storage allocated, the problem of dangling reference arises. The dangling reference occurs when there is a reference to storage that has been deallocated.
- It is a logical error to a dangling reference, since, the value of de use de- allocated storage is undefined according to the semantics of most languages.
- Whenever storage allocated, the problem of dangling reference arises. The dangling reference occurs when there is a reference to storage that has been allocated

Stack Allocation

- Variable length data



Access to dynamically allocated arrays

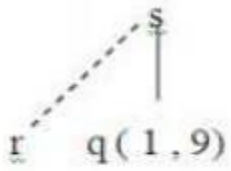
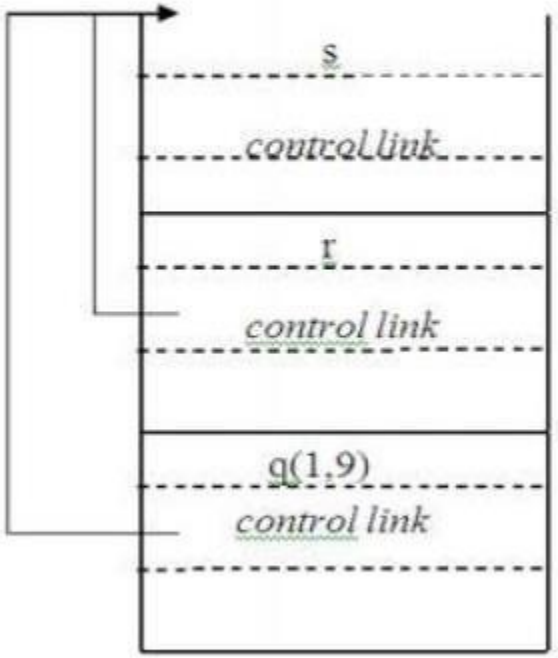
Heap Allocation

- Stack allocation strategy cannot be used if either of the following is possible :

1. The values of local names must be retained when an activation ends.
2. A called activation outlives the caller.

- Heap allocation parcels out pieces of contiguous storage, as needed for activation records or other objects.

Heap Allocation

Position in the activation tree	Activation records in the heap	Remarks
		Retained activation record for r

Intermediate code Generator

INTERMEDIATE CODE GENERATION (ICG)

- In compiler, the front-end translates a source program into an intermediate representation from which the back end generates target code.

Need For ICG

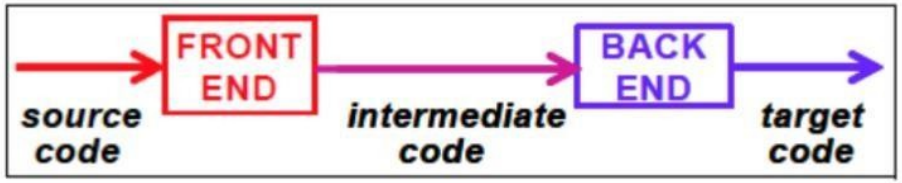
- If a compiler translates the source language to its target machine language without generating IC, then for each new machine, a full native compiler is required.
- IC eliminates the need of a new full compiler for every machine by keeping the analysis portion for all the compilers.
- Synthesis part of back end depends on the target machine.

Benefits of ICG

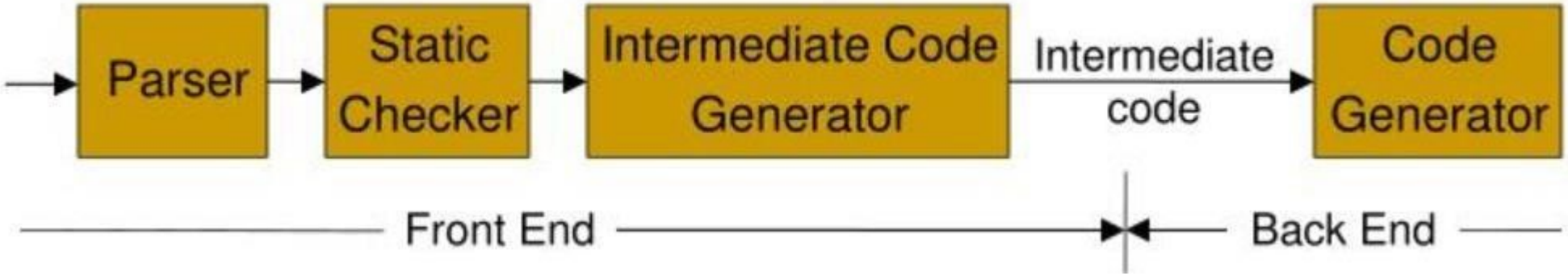
- A source program can be translated directly into the target language, but some benefits of using intermediate form are:
  - Retargeting is facilitated: a compiler for a different machine can be created by attaching a Back-end(which generate Target Code) for the new machine to an existing Front-end (which generate Intermediate Code).
  - A machine Independent Code-Optimizer can be applied to the Intermediate Representation.

2 important things

- IC Generation process should not be very complex
- It shouldn't be difficult to produce the target program from the intermediate code.



Logical Structure of a Compiler Front End



INTERMEDIATE LANGUAGES

- The most commonly used intermediate representations were :-
    - Syntax Tree
    - DAG (Direct Acyclic Graph)
    - Postfix Notation
    - Three Address Code

GRAPHICAL REPRESENTATION

Includes both

- Syntax Tree
- DAG (Direct Acyclic Graph)

Syntax Tree Or Abstract Syntax Tree(AST)

- Graphical Intermediate Representation
- Syntax Tree depicts the hierarchical structure of a source program.

- Syntax tree (AST) is a condensed form of parse tree useful for representing language constructs.

Syntax Tree Or Abstract Syntax Tree(AST)

<u>EXAMPLE</u>

- Construct parse tree and syntax tree for **3 * 5 + 4** as follows.

Syntax Tree Or Abstract Syntax Tree(AST)

Grammar

E → E +T

E → E - T

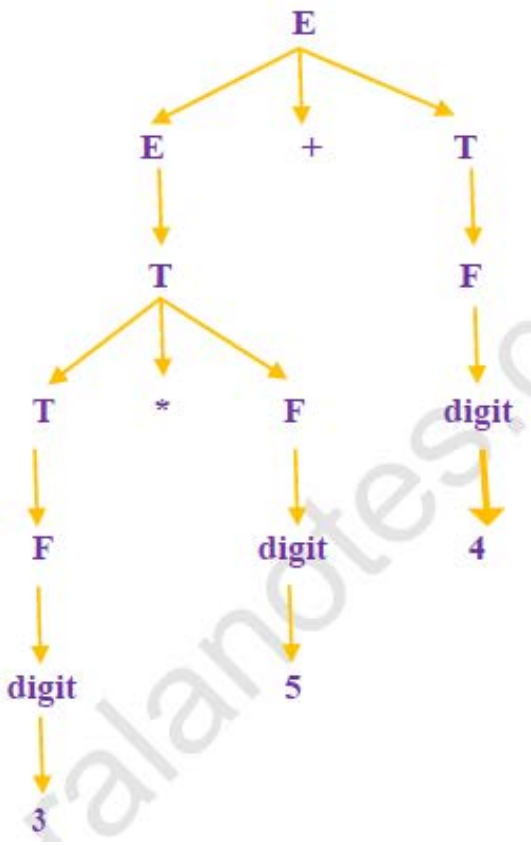
E→ T

T→T * F

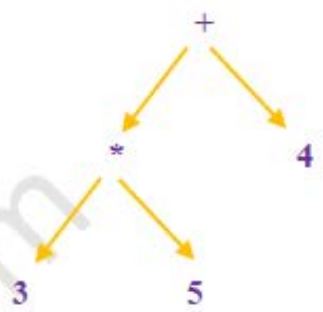
T→F

F→ digit

Parse Tree



Syntax Tree



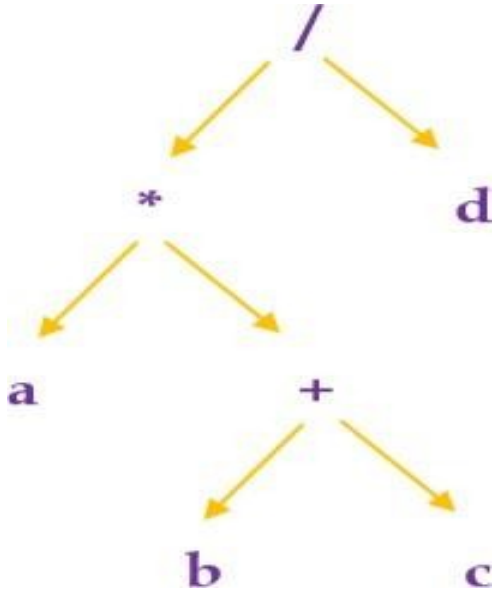
ralanotes.com

Parse Tree VS Syntax Tree

Parse Tree	Syntax Tree
A parse tree is a graphical representation of a replacement process in a derivation	A syntax tree (AST) is a condensed form of parse tree
Each interior node represents a grammar rule	Each interior node represents an operator
Each leaf node represents a terminal	Each leaf node represents an operand
Parse tree represent every detail from the real syntax	Syntax tree does not represent every detail from the real syntax Eg : No parenthesis

Example

- Syntax tree for a * (b + c) /d



Constructing Syntax Tree For Expression

- Each node in a syntax tree can be implemented in a record with several fields.
- In the node of an operator, one field contains operator and remaining field contains pointer to the nodes for the operands.
- When used for translation, the nodes in a syntax tree may contain addition of fields to hold the values of attributes attached to the node.

Constructing Syntax Tree For Expression

Following functions are used to create syntax tree

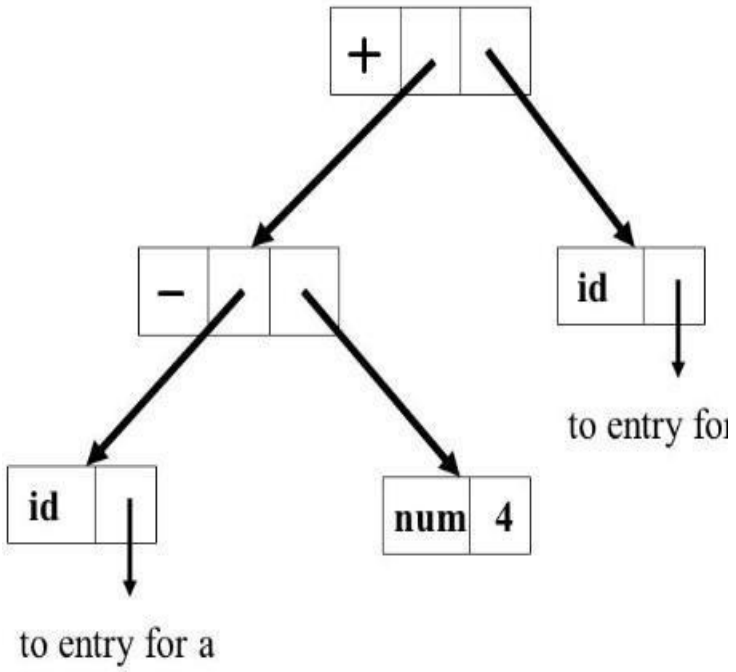
- **mknode(op,left,right)**: creates an operator node with label op and two fields containing pointers to left and right.
- **mkleaf(id,entry)**: creates an identifier node with label id and a field containing entry, a pointer to the symbol table entry for identifier
- **mkleaf(num,val)**: creates a number node with label num and a field containing val, the value of the number.

Such functions return a pointer to a newly created node.

Constructing Syntax Tree For Expression

Construct syntax tree for a – 4 + c

- P1 = **mkleaf(id,entry a)**
- P2 = **mkleaf(num, 4)**
- P3 = **mknode(-, P1, P2)**
- P4 = **mkleaf(id,entry c)**
- P5 = **mknode(+, P3, P4)**



Direct Acyclic Graph (DAG)

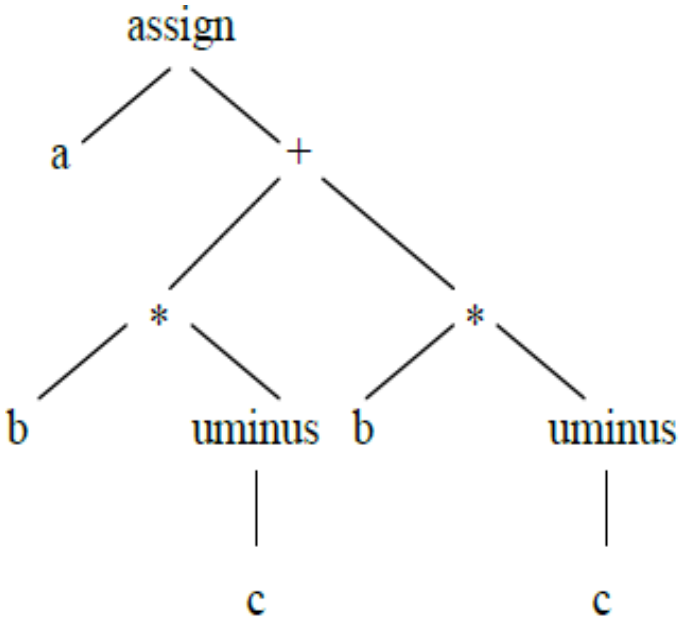
- Directed Acyclic Graphs|Compiler Design|Part# 42|Learn CSE Malayalam

Direct Acyclic Graph (DAG)

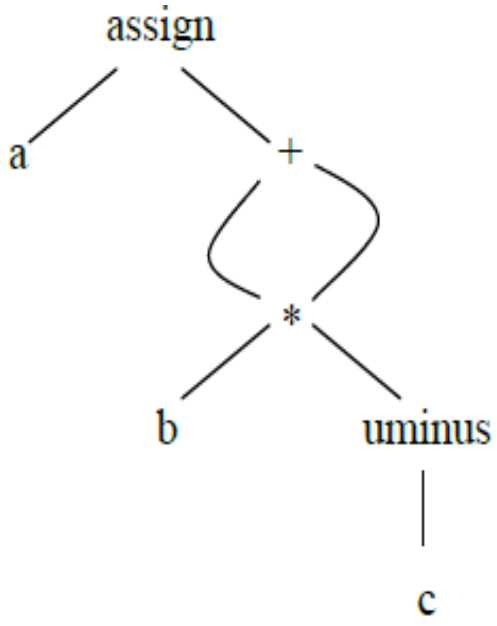
- Graphical Intermediate Representation
- DAG also gives the hierarchical structure of source program but in a more compact way because common sub expressions are identified

DAG Example

a=b*-c + b*-c



(a) Syntax tree

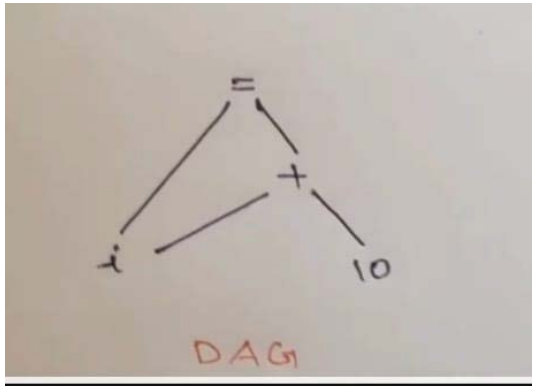


(b) Dag

DAG

The value number method for constructing DAG

Example : i = i + 10



1	id	entry for i	
2	num	10	
3	+	1	2
4	=	1	3
5	……		

Postfix Notation

- Linearized representation of syntax tree
- In postfix notation, each operator appears immediately after its last operand.
- Operators can be evaluated in the order in which they appear in the string

Postfix Notation - Example

- Source String :
  - a := b * -c + b * -c
- Postfix String:
  - a b c uminus * b c uminus * + assign

Postfix Rules

- If E is a variable or constant, then the postfix notation for E is E itself.
- If E is an expression of the form E1 op E2 then postfix notation for E is E1’ E2’ op, here E1’ and E2’ are the postfix notations for E1and E2, respectively
- If E is an expression of the form (E), then the postfix notation for E is the same as the postfix notation for E.
- For unary operation –E the postfix is E-

Postfic Notation - Example

- Ex: postfix notation for 9- (5+2) is

  952+-

- Postfix notation of an infix expression can be obtained using stack

Three address Code

- Intermediate Code Generation|Compiler Design|Part# 43|Learn CSE Malayalam

Three address Code

- In Three address statement, at most 3 addresses are used to represent any statement.
- The reason for the term “three address code” is that each statement contains 3 addresses at most. Two for the operands and one for the result.

Three address Code

General Form of 3 Address Code

$$a = b \text{ op } c$$

where,

- a, b, c are the operands that can be names, constants or compiler generated temporaries.
- op represents operator, such as fixed or floating point arithmetic operator or a logical operator on Boolean valued data

Three address Code

- Thus a source language expression like x + y * z might be translated into a sequence
- t1 := y*z
- t2 := x+t1
- where, t1 and t2 are compiler generated temporary names.

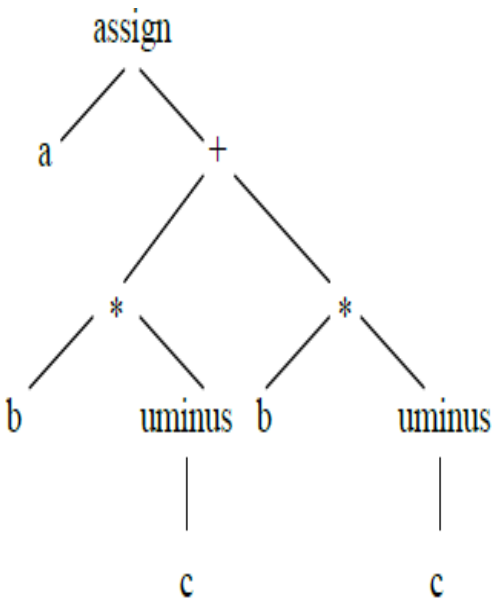
Advantages of Three Address Code

1. The unraveling of complicated arithmetic expressions and of nested flow-of-control statements makes three-address code desirable for target code generation and optimization.
2. The use of names for the intermediate values computed by a program allows three- address code to be easily rearranged - unlike postfix notation.

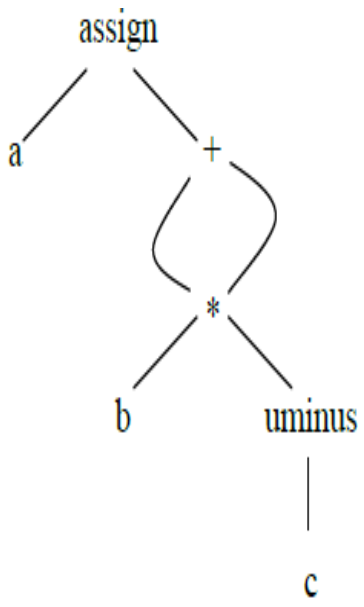
- Three-address code is a linearized representation of a syntax tree or a dag in which explicit names correspond to the interior nodes of the graph

Example

- Three Address Code corresponding to the syntax tree and DAG for the expression a=b*-c + b*-c



(a) Syntax tree



(b) Dag

$t_1 := -c$

$t_2 := b * t_1$

$t_3 := -c$

$t_4 := b * t_3$

$t_5 := t_2 + t_4$

$a := t_5$

(a) Code for the syntax tree

$t_1 := -c$

$t_2 := b * t_1$

$t_5 := t_2 + t_2$

$a := t_5$

(b) Code for the dag

Types of Three-Address Statements

Assignment statements

- x := y op z, where op is a binary arithmetic or logical operation.

Assignment instructions

- x : = op y, where op is a unary operation . Essential unary operations include unaryminus, logical negation, shift operators, and conversion operators that for example, convert a fixed-point number to a floating-point number.

Copy statements

- x : = y where the value of y is assigned to x.

Types of Three-Address Statements

Unconditional jump

- goto L The three-address statement with label L is the next to be executed

Conditional jump

- if x relop y goto L
- This instruction applies a relational operator (<, =, =, etc,) to x and y, and executes the statement with label L next if x stands in relation relop to y. If not, the three-address statement following if x relop y goto L is executed next, asin the usual sequence.

Types of Three-Address Statements

Procedural call and return

- param x and call p, n for procedure calls and return y, where y representing a returned value is optional. Their typical use is as the sequence of three-address statements

param x1

param x2

…….

param xn

call p,n

- generated as part of the call procedure p(x1 , x2, . . . , xn) . The integer n indicating the number of actual-parameters in "call p , n" is not redundant because calls can be nested.

Types of Three-Address Statements

Indexed Assignments

- Indexed assignments of the form x = y[i] or x[i] = y

Address and pointer assignments

- Address and pointer operator of the form
- x := &y,
- x := *y

and

- *x := y

Implementation Of Three-Address Statements

- A three address statement is an abstract form of intermediate code. In a compiler, these statements can be implemented as records with fields for the operator and the operands. Three such, representations are
- Quadruples
- Triples
- Indirect triples

QUADRUPLES

- A quadruple is a record structure with four fields, which are op, arg1, arg2 and result
- The op field contains an internal code for the operator. The three address statement x:= y op z is represented by placing y in arg1, z in arg2 and x in result.
- The contents of arg1, arg2, and result are normally pointers to the symbol table entries for the names represented by these fields. If so temporary names must be entered into the symbol table as they are created

QUADRUPLES - Example

- Translate the following expression to quadruple

  a + b * c | e ^ f + b * a

- For the first construct the three address code for the expression
- t1 = e ^ f
- t2 = b * c
- t3 = t2 / t1
- t4 = b * a
- t5 = a + t3
- t6 = t5 + t4

QUADRUPLES - Example

Location	OP	arg1	arg2	Result
(0)	^	e	f	t1
(1)	*	b	c	t2
(2)	/	t2	t1	t3
(3)	*	b	a	t4
(4)	+	a	t3	t5
(5)	+	t3	t4	t6

t1 = e ^ f
t2 = b * c
t3 = t2 / t1
t4 = b * a
t5 = a + t3
t6 = t5 + t4

TRIPLES

- In triples representation, the use of temporary variables is avoided & instead reference to instructions are made
- So three address statements can be represented by records with only three fields OP, arg1 & arg2.
- Since, three fields are used this intermediated code formal is known as triples

TRIPLES

Advantages

- No need to use temporary variable which saves memory as well as time

Disadvantages

- Triple representation is difficult to use in optimizing compilers.
- During optimization, statements may need to be reordered.
- In triples, results are referred to using statement numbers (indexes).
- If the order of statements changes, these references must also be updated, making optimization difficult.

TRIPLES – Example 1

Translate the following expression to triples

a + b * c | e ^ f + b * a

Location	OP	arg1	arg2
(0)	^	e	f
(1)	*	b	c
(2)	/	(1)	(0)
(3)	*	b	a
(4)	+	a	(2)
(5)	+	(4)	(3)

t1 = e ^ f
t2 = b * c
t3 = t2 / t1
t4 = b * a
t5 = a + t3
t6 = t5 + t4

TRIPLES – Example 2

- A ternary operation like x[i] : = y requires two entries in the triple structure while x : = y[i] is naturally represented as two operations.

	op	*arg1*	*arg2*
(0)	[ ] =	x	i
(1)	assign	(0)	y

x[i] := y

	op	*arg1*	*arg2*
(0)	= [ ]	y	i
(1)	assign	x	(0)

x := y[i]

INDIRECT TRIPLES

- This representation is an enhancement over triple representation.
- It uses an additional instruction array to led the pointer to the triples in the desired order.
- Since, it uses pointers instead of position to stage reposition the expression to produces optimized code

INDIRECT TRIPLES – Example 1

	Statement
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

location	op	arg1	arg2
(0)	^	e	f
(1)	*	b	c
(2)	/	(1)	(0)
(3)	*	b	a
(4)	+	a	(2)
(5)	+	(4)	(3)

Comparison

- When we ultimately produce the target code each temporary and programmer defined name will assign runtime memory location
- This location will be entered into symbol table entry of that data.
- Using the quadruple notation, a three address statement containing a temporary can immediately access the location for that temporary via symbol table.
- But this is not possible with triples notation
- With quadruple notation, statements can often move around which makes optimization easier.

Comparison

- This is achieved because using quadruple notation the symbol table interposes high degree of indirection between computation of a value and its use.
- With quadruple notation, if we move a statement computing x, the statement using x requires no change.
- But with triples, moving a statement that defines a temporary value requires us to change all references to that statement in arg1 and arg2 arrays. This makes triples difficult to use in optimizing compiler
- With indirect triples also, there is no such problem.
- A statement can be moved by reordering the statement list

Space Utilization

- Quadruples and indirect triples requires same amount of space for storage (normal case).
- But if same temporary value is used more than once indirect triples can save some space. This is bcz, 2 or more entries in statement array can point to the same line of op-arg1- arg2 structure.
- Triples requires less space for storage compared to above 2.

Comparison

Quadruples

- direct access of the location for temporaries
- easier for optimization

Triples

- space efficiency

Indirect Triples

- easier for optimization
- space efficiency

PROBLEM 1

- Translate the following expression to quadruple, triples & indirect triples

a = b * - c + b * - c

Sol : - Three address code for given expression is

- t1 = uniminus c
- t2 = b* t1
- t3 = uniminus c
- t4 = b* t3
- t5 = t2 + t4
- a = t5

QUADRUPLES

Location	OP	Arg1	Arg2	Result
(0)	uniminus	c		t1
(1)	*	b	t1	t2
(3)	uniminus	c		t3
(4)	*	b	t3	t4
(5)	+	t2	t4	t5
(6)	=	t5		a

TRIPLES

Location	OP	arg1	arg2
(1)	uniminus	c	
(2)	*	b	(1)
(3)	uniminus	c	
(4)	*	b	(3)
(5)	+	(2)	(4)
(6)	=	a	(5)

INDIRECT TRIPLE

	Statements
35	(1)
36	(2)
37	(3)
38	(4)
39	(5)
40	(6)

Location	OP	arg1	arg2
(1)	uniminus	c	
(2)	*	b	(1)
(3)	uniminus	c	
(4)	*	b	(3)
(5)	+	(2)	(4)
(6)	=	a	(5)